

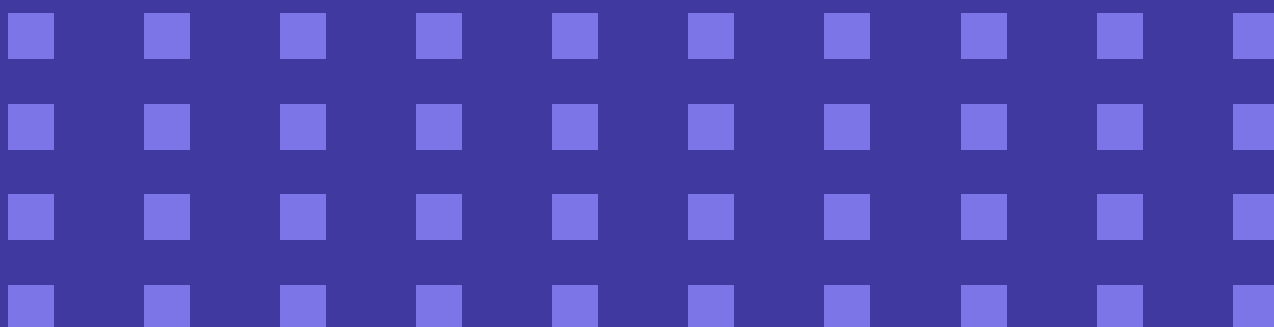
COMPILATEUR JAVASCRIPT VERS COMMODORE 64

JS-C64

Guide pratique du débutant

Compiler un fichier .js en PRG, ASM, LST, BASIC ou D64
et construire progressivement un petit jeu C64.

Édition condensée - API 1.0



Comment utiliser ce guide

Ce document est une version condensée et progressive de **MODE_EMPLOI_DEBUTANT.txt**. Il est conçu pour être lu dans l'ordre une première fois, puis utilisé comme aide-mémoire.

Le principe en une phrase

Vous écrivez un fichier JavaScript composé d'appels à **c64**. Le compilateur exécute ce fichier pendant le build et produit directement du code machine 6502 pour le Commodore 64.

Étape	Ce que vous faites	Résultat
1	Écrire mon-jeu.js	Description du programme avec l'API c64
2	Lancer c64js build	Compilation sur le PC avec Node.js
3	Choisir l'extension de sortie	PRG, ASM, LST, BAS, BIN ou D64
4	Ouvrir la sortie dans VICE ou sur un C64	Le code 6502 s'exécute sans JavaScript

Table des matières

Comment utiliser ce guide	2
Table des matières	3
1. Comprendre JS-C64	6
1.1 Ce que fait le compilateur	6
1.2 Coordonnées et valeurs importantes	6
2. Installer et accéder à c64js	6
2.1 Préparation dans le dépôt du compilateur	6
2.2 Méthode recommandée dans ce dépôt	6
2.3 Dans votre propre projet avec npm	6
2.4 Installation locale avant publication	7
2.5 Commande globale facultative	7
3. Compiler vers PRG, ASM, BAS, LST, BIN et D64	7
3.1 Les formats de sortie	7
3.2 Commandes prêtes à copier	7
3.3 Options utiles	8
3.4 Produire une D64 multi-niveaux	8
4. Premier programme pas à pas	8
4.1 Créer le fichier JavaScript	8
4.2 Compiler et lancer	8
4.3 Ce que chaque ligne signifie	9
5. Cours pratique - un mini-jeu au joystick	9
5.1 Le programme complet	9
5.2 Comprendre les quatre étapes	10
5.3 Compiler le mini-jeu	10
6. Affichage, couleurs et mémoire	10
6.1 Fonctions d'écran essentielles	10
6.2 Les 16 couleurs	11

6.3 Mémoire et données	11
7. Boucle de jeu, contrôles et entrées	11
7.1 Structure conseillée	11
7.2 Contrôle runtime	12
7.3 Joystick et clavier	12
8. Sprites, animations et collisions	12
8.1 Créer un sprite	12
8.2 Méthodes principales	13
8.3 Les sprites virtuels 8 à 15	13
9. Charsets, tiles, maps et scrolling	13
9.1 Charger une map JSON	13
9.2 Lire et modifier la map	13
9.3 Objets de map et caméra	14
9.4 Collisions de plateforme	14
10. Scènes et jeux multi-niveaux	14
10.1 Une scène par état du jeu	14
10.2 Stratégie mémoire D64	14
11. Son SID	15
11.1 Réglages directs	15
11.2 Musique et bruitages non bloquants	15
12. IRQ raster et assembleur bas niveau	15
12.1 IRQ raster	15
12.2 Assembleur 6502 dans le JavaScript	16
13. Optimisation et débogage	16
13.1 Choisir le profil	16
13.2 Lire les sorties de diagnostic	16
13.3 Erreurs courantes	17

14. Référence rapide des fonctions	17
14.1 Écran et système	17
14.2 Données, variables et contrôle	17
14.3 Entrées, sprites et collisions	18
14.4 Assets, maps et caméra	18
14.5 Son, IRQ et assembleur	18
15. Méthode conseillée pour votre premier jeu	18
15.1 Progression en sept petites étapes	18
15.2 Checklist avant de continuer	19

1. Comprendre JS-C64

1.1 Ce que fait le compilateur

JS-C64 n'embarque pas un moteur JavaScript dans le C64. Le fichier source est exécuté par Node.js au moment de la compilation. Chaque appel à l'objet **c64** demande au compilateur de générer des instructions 6502, des tables de données ou des routines runtime.

- Le fichier **.js** est le source lisible et facile à modifier.
- Le fichier **.prg** contient le programme exécutable par le C64.
- Le C64 final n'a besoin ni de Node.js, ni de npm, ni du fichier JavaScript.
- Seules les fonctions réellement utilisées ajoutent leur runtime au programme final.

1.2 Coordonnées et valeurs importantes

Élément	Valeur	À retenir
Écran texte	40 x 25 caractères	x va de 0 à 39, y de 0 à 24
Pixel d'un sprite	24 x 21 en hires	La coordonnée X du VIC-II utilise 9 bits
Couleurs	0 à 15	Utilisez les constantes <code>c64.COLOR_*</code>
Octet	0 à 255	Pour les valeurs signées, le compilateur gère les vitesses négatives
Frame PAL	50 Hz	Une boucle de jeu est généralement appelée 50 fois par seconde

Important

Le code exécuté pendant la compilation doit être fiable. Un fichier JS-C64 est du vrai JavaScript côté PC, même si son objectif est de produire du code 6502.

2. Installer et accéder à c64js

2.1 Préparation dans le dépôt du compilateur

Cette méthode est la plus fiable pendant le développement de js-c64, notamment si Windows indique que la commande **c64js** n'est pas reconnue.

```
cd "D:\Programmation\Vibe Coding Pro\compilateur js-c64"
Get-Location
Get-ChildItem package.json
node -v
npm -v
npm install
```

Les guillemets autour du chemin sont obligatoires parce qu'il contient des espaces. `npm install` est normalement nécessaire une seule fois, puis après une modification des dépendances.

2.2 Méthode recommandée dans ce dépôt

```
node .\src\cli.js build .\examples\hello.js -o .\dist\hello.prg
```

Cette commande appelle directement le programme de compilation. Elle fonctionne même si `c64js` n'est pas installé globalement.

2.3 Dans votre propre projet avec npm

```
npm install js-c64
npx c64js build .\examples\hello.js -o .\build\hello.prg
```

npx recherche la commande installée localement dans node_modules/.bin. C'est la méthode conseillée pour un projet utilisateur reproductible.

2.4 Installation locale avant publication

```
npm install "D:\Programmation\Vibe Coding Pro\compilateur js-c64"
npx c64js build .\examples\hello.js -o .\build\hello.prg
```

2.5 Commande globale facultative

```
# À lancer depuis le dossier de js-c64
npm link
Get-Command c64js
c64js build .\examples\hello.js -o .\dist\hello.prg
```

Si c64js n'est pas reconnu

Revenez à la commande fiable `node .\src\cli.js build ...`, ou utilisez `npx c64js` dans un projet où le package est installé.

3. Compiler vers PRG, ASM, BAS, LST, BIN et D64

3.1 Les formats de sortie

Extension	Contenu	Usage principal
.prg	Exécutable C64 avec adresse de chargement	Charger dans VICE puis RUN
.asm	Assembleur 6502 lisible	Étudier, déboguer ou reprendre le code
.lst	Listing avec adresses et octets	Relier le PC du C64 à une instruction précise
.bas	Chargeur BASIC et lignes DATA	Copier ou conserver le programme sous forme BASIC
.bin	Code machine brut	Intégration avancée sans en-tête PRG
.d64	Image de disquette 1541	Jeux multi-niveaux avec assets chargés à la demande

3.2 Commandes prêtes à copier

```
# PRG exécutable
node .\src\cli.js build .\examples\hello.js -o .\dist\hello.prg

# Assembleur lisible
node .\src\cli.js build .\examples\hello.js -o .\dist\hello.asm --format asm

# Listing avec adresses
node .\src\cli.js build .\examples\hello.js -o .\dist\hello.lst --format lst

# BASIC avec DATA
node .\src\cli.js build .\examples\hello.js -o .\dist\hello.bas --format data

# Binaire brut
node .\src\cli.js build .\examples\hello.js -o .\dist\hello.bin --format bin
```

Le format est déduit de l'extension. --format reste utile pour rendre la commande explicite.

3.3 Options utiles

Option	Exemple	Effet
--sys	--sys 49152	Choisit l'adresse de départ, surtout pour BAS, ASM, LST et BIN
--opt	--opt balanced	balanced par défaut, size pour la taille, speed pour la vitesse
--map	--map symbols.json	Écrit les symboles et leurs adresses
--report	--report rapport.json	Détaille mémoire, assets, routines et optimisations
--assets	--assets disk	Place les assets de niveaux dans des fichiers disque
--device	--device 8	Numéro du lecteur utilisé au runtime
--disk-name	--disk-name "MON JEU"	Nom de l'image D64
--program-name	--program-name START	Nom du PRG principal sur la disquette

3.4 Produire une D64 multi-niveaux

```
node .\src\cli.js build .\examples\multilevel-d64.js `
-o .\dist\multilevel.d64

node .\src\cli.js build .\examples\multilevel-d64.js `
-o .\dist\jeu.d64 `
--disk-name "MON JEU" --program-name START --device 8
```

L'extension **.d64** active automatiquement les assets disque. L'image contient un PRG principal et des modules PRG de données chargés par le KERNAL lorsque `level.activate()` ou une transition de niveau en a besoin.

PRG autonome ou D64

Le même JavaScript peut produire un PRG autonome avec `--assets inline`, ou une D64 plus adaptée aux nombreux niveaux avec les assets externes.

```
# Runtime disque lisible en assembleur
node .\src\cli.js build .\examples\multilevel-d64.js `
-o .\dist\multilevel.asm --format asm --assets disk

# Version PRG entièrement autonome
node .\src\cli.js build .\examples\multilevel-d64.js `
-o .\dist\multilevel.prg --assets inline
```

4. Premier programme pas à pas

4.1 Créer le fichier JavaScript

Créez par exemple `examples/bonjour.js` :

```
import { c64 } from "js-c64";

c64.clearScreen();
c64.borderColor(c64.COLOR_BLUE);
c64.backgroundColor(c64.COLOR_BLUE);
c64.textColor(c64.COLOR_WHITE);
c64.printCentered(12, "BONJOUR COMMODORE 64");
```

4.2 Compiler et lancer

```
npx c64js build .\examples\bonjour.js -o .\build\bonjour.prg
```

- Ouvrez **bonjour.prg** dans VICE.
- Si l'émulateur ne lance pas automatiquement le programme, tapez RUN.
- Pour comprendre le résultat, produisez aussi **bonjour.asm** et **bonjour.lst**.

4.3 Ce que chaque ligne signifie

Code	Rôle
import { c64 } ...	Donne accès à l'API pendant la compilation
clearScreen()	Efface les 1 000 cellules de l'écran texte
borderColor()	Change la couleur de la bordure VIC-II
backgroundColor()	Change le fond de l'écran
textColor()	Choisit la couleur des prochains textes
printCentered()	Calcule automatiquement la colonne de départ

5. Cours pratique - un mini-jeu au joystick

L'exemple suivant a été compilé pendant la création de ce guide. Il montre une boucle de jeu, un sprite, le joystick du port 2, un score et un bruitage non bloquant.

5.1 Le programme complet

```
import { c64 } from "js-c64";
const PLAYER_PIXELS = Array.from(
  { length: 63 },
  (_, index) => (index < 54 && index % 3 === 1 ? 0x7e : 0)
);
const joystick = c64.input.joystick(2);
const score = c64.game.score({ name: "demoScore", digits: 3, initial: 0 });
const player = c64.sprite.create(0, {
  x: 150,
  y: 120,
  data: PLAYER_PIXELS,
  color: c64.COLOR_YELLOW,
  minX: 24,
  maxX: 296,
  minY: 50,
  maxY: 220
});
c64.game.init(() => {
  c64.clearScreen();
  c64.borderColor(c64.COLOR_BLACK);
  c64.backgroundColor(c64.COLOR_BLUE);
  c64.textColor(c64.COLOR_WHITE);
  c64.printCentered(2, "MON PREMIER MINI JEU");
  c64.printAt(2, 23, "JOY 2: BOUGER FIRE: SCORE");
  c64.printAt(29, 2, "SCORE");
  score.draw(35, 2, { color: c64.COLOR_YELLOW });
});
c64.game.frame(() => {
  player.setVelocity(0, 0);
  c64.control.if(joystick.left(), () => player.setVelocity(-2, 0));
  c64.control.if(joystick.right(), () => player.setVelocity(2, 0));
  c64.control.if(joystick.up(), () => player.setVelocity(0, -2));
  c64.control.if(joystick.down(), () => player.setVelocity(0, 2));
  c64.control.if(joystick.firePressed(), () => {
    score.inc();
    score.draw(35, 2, { color: c64.COLOR_YELLOW });
    c64.sid.click();
  });
  player.update();
}, { hz: 50 });
```

5.2 Comprendre les quatre étapes

Étape A - Les ressources de compilation

PLAYER_PIXELS est calculé par JavaScript sur le PC. Le résultat devient 63 octets de données sprite dans le PRG.

Étape B - Les objets runtime

joystick, score et player produisent des variables et routines 6502. Ils ne sont pas des objets JavaScript présents sur le C64.

Étape C - L'initialisation

Le contenu de `c64.game.init()` est exécuté une seule fois : préparation de l'écran et dessin initial du score.

Étape D - La frame

Le contenu de `c64.game.frame()` est exécuté 50 fois par seconde. La vitesse est remise à zéro, modifiée selon le joystick, puis `player.update()` applique le mouvement et les limites.

5.3 Compiler le mini-jeu

```
npx c64js build .\examples\mini-jeu.js -o .\build\mini-jeu.prg
npx c64js build .\examples\mini-jeu.js -o .\build\mini-jeu.asm --format asm
npx c64js build .\examples\mini-jeu.js -o .\build\mini-jeu.lst --format lst
```

Exercice

Ajoutez une touche pause, changez la vitesse de 2 à 3, puis remplacez le bruitage `sid.click()` par un autre effet. Recompilez après chaque petite modification.

6. Affichage, couleurs et mémoire

6.1 Fonctions d'écran essentielles

Fonction	Description	Exemple
<code>c64.clearScreen()</code>	Efface l'écran texte	<code>c64.clearScreen();</code>
<code>c64.borderColor(c)</code>	Couleur de bordure	<code>c64.borderColor(c64.COLOR_BLACK);</code>
<code>c64.backgroundColor(c)</code>	Couleur de fond	<code>c64.backgroundColor(c64.COLOR_BLUE);</code>
<code>c64.textColor(c)</code>	Couleur des textes suivants	<code>c64.textColor(c64.COLOR_WHITE);</code>
<code>c64.print(text)</code>	Écrit à la position courante	<code>c64.print("READY");</code>
<code>c64.printAt(x,y,text)</code>	Écrit à une cellule précise	<code>c64.printAt(2, 5, "SCORE");</code>
<code>c64.printCentered(y,text)</code>	Centre sur 40 colonnes	<code>c64.printCentered(12, "PAUSE");</code>
<code>c64.writeChar(x,y,ch,c)</code>	Écrit un code écran	<code>c64.writeChar(10, 10, 64, 7);</code>
<code>c64.fillRect(...)</code>	Remplit un rectangle	<code>fillRect(1,1,10,5,32,1)</code>
<code>c64.drawFrame(...)</code>	Dessine un cadre	<code>drawFrame(1,1,38,8,1,1)</code>
<code>c64.clearLine(y,...)</code>	Nettoie une ligne	<code>c64.clearLine(20);</code>

6.2 Les 16 couleurs

Constante	N°	Constante	N°
COLOR_BLACK	0	COLOR_WHITE	1
COLOR_RED	2	COLOR_CYAN	3
COLOR_VIOLET	4	COLOR_GREEN	5
COLOR_BLUE	6	COLOR_YELLOW	7
COLOR_ORANGE	8	COLOR_BROWN	9
COLOR_LIGHTRED	10	COLOR_DARKGREY	11
COLOR_GREY	12	COLOR_LIGHTGREEN	13
COLOR_LIGHTBLUE	14	COLOR_LIGHTGREY	15

6.3 Mémoire et données

API	Utilité
c64.poke(adresse, valeur)	Écrit un octet à une adresse
c64.peek(adresse)	Lit un octet pour une opération générée
c64.memset(adresse, valeur, longueur)	Remplit une zone mémoire
c64.memcpy(destination, source, longueur)	Copie une zone mémoire
c64.copyDataTo(adresse, dataRef, longueur)	Copie une donnée nommée
c64.data.byte/word/string	Déclare des données constantes
c64.data.screenString	Prépare une chaîne en codes écran
c64.dataRef(nom)	Référence une donnée déclarée
c64.var.byte/word/bool	Déclare un état mutable au runtime
c64.table.byte(nom, valeurs)	Crée un tableau d'octets runtime

7. Boucle de jeu, contrôles et entrées

7.1 Structure conseillée

```
const joystick = c64.input.joystick(2);
const active = c64.var.bool("active", { initial: true });

c64.game.init(() => {
  // Une seule fois
});

c64.game.frame(() => {
  // À chaque image logique
  c64.control.if(joystick.firePressed(), () => active.toggle());
}, { hz: 50 });
```

7.2 Contrôle runtime

Fonction	Rôle
c64.control.if(condition, oui, non?)	Génère un branchement runtime
c64.control.while(condition, corps, {maxIterations})	Boucle bornée obligatoire
c64.control.routine(nom, fonction)	Déclare une sous-routine partagée
c64.control.call(nom)	Appelle une routine déclarée
c64.game.every(n, fonction)	Exécute une action toutes les n frames
variable.set/inc/dec/add/sub/toggle	Modifie une variable runtime
variable.eq/ne/lt/lte/gt/gte	Construit une condition runtime

7.3 Joystick et clavier

API	Valeur
joystick.left/right/up/down()	Vrai tant que la direction est maintenue
joystick.fire()	Vrai tant que FIRE est maintenu
joystick.firePressed()	Vrai une seule frame au moment de l'appui
joystick.fireReleased()	Vrai une seule frame au relâchement
c64.input.keyboard({ pause: c64.KEY_SPACE })	Déclare les touches à surveiller
keys.pause.pressed()	Détecte un nouvel appui sans répétition

Piège fréquent

N'utilisez pas une condition JavaScript normale pour tester une variable runtime. Utilisez
`c64.control.if(variable.eq(...), ...)`.

8. Sprites, animations et collisions

8.1 Créer un sprite

```
const player = c64.sprite.create(0, {  
  x: 100,  
  y: 100,  
  data: SPRITE_63_OCTETS,  
  color: c64.COLOR_YELLOW,  
  multicolor: false,  
  expandX: false,  
  expandY: false,  
  hitbox: { offsetX: 0, offsetY: 0, width: 24, height: 21 },  
  minX: 24,  
  maxX: 296  
});
```

8.2 Méthodes principales

Méthode	Description
<code>sprite.setPosition(x, y)</code>	Positionne le sprite, y compris au-delà de X=255
<code>sprite.setVelocity(vx, vy)</code>	Définit une vitesse signée
<code>sprite.update()</code>	Applique mouvement, animation, limites et affichage
<code>sprite.play(animation)</code>	Choisit une animation nommée sans la redémarrer inutilement
<code>sprite.collides(other)</code>	Collision AABB logique
<code>sprite.enable()/disable()</code>	Affiche ou masque le sprite logique
<code>sprite.respawn()</code>	Réinitialise l'entité lorsqu'elle vient d'une map

8.3 Les sprites virtuels 8 à 15

Créer le sprite 8 à 15 active automatiquement le multiplexeur. Le runtime trie les sprites par Y et réutilise les huit canaux matériels lorsque le raster avance.

- Les 16 sprites logiques peuvent être placés librement en X et Y.
- Le VIC-II ne peut jamais afficher plus de huit sprites sur une même ligne raster.
- Un sprite agrandi verticalement occupe davantage de lignes et réduit les possibilités de réutilisation.
- Avec le multiplexage, préférez les collisions logicielles `collides()`.
- Une boucle `c64.game.frame()` est obligatoire.

9. Charsets, tiles, maps et scrolling

9.1 Charger une map JSON

```
const level = c64.assets.loadMap("assets/mon-niveau.json");

c64.game.init(() => {
  c64.charset.use(level.charset, { address: 0x3000 });
  c64.map.draw(level, { x: 0, y: 0 });
});
```

Les codes écran 0 à 63 sont réservés automatiquement aux caractères originaux du C64. Le compilateur les copie depuis la ROM vers la RAM dès qu'un charset personnalisé est installé. Le Studio n'exporte donc que les glyphes personnalisés, qui commencent au code 64.

9.2 Lire et modifier la map

```
const tileX = c64.var.byte("tileX", { initial: 4 });
const tileY = c64.var.byte("tileY", { initial: 7 });

c64.control.if(level.map(tileX, tileY).isSolid(), () => {
  level.map(tileX, tileY).set(0);
});
```

Opération	Description
<code>level.map(x, y).set(tile)</code>	Modifie la map et redessine la cellule visible
<code>.load(variable)</code>	Charge l'indice de tile dans une variable
<code>.eq/.ne(tile)</code>	Compare la cellule
<code>.isSolid()</code>	Teste la collision logique non nulle/solide
<code>.hasCollision(valeur)</code>	Teste une classe précise de collision

Opération	Description
level.activate({draw:true})	Active et dessine un niveau, y compris depuis D64
level.isActive()	Condition runtime indiquant le niveau actif

9.3 Objets de map et caméra

```
const player = c64.map.spawn(level, "player", {
  sprite: 0,
  maxCollisionSpeed: 8,
  collisionBehaviors: { 1: "solid", 2: "danger" }
});

const camera = c64.map.scroller(level, {
  width: 38,
  x: 1,
  panel: { bottom: 3 }
});

camera.follow(player);
```

Le scroller gère le déplacement fin X/Y, la copie des nouvelles lignes ou colonnes et une bande IRQ pour garder un panneau fixe. panel: "bottom" conserve la compatibilité demi-écran ; { bottom: 3 } réserve exactement trois lignes.

9.4 Collisions de plateforme

Comportement	Effet
solid	Bloque l'entité dans toutes les directions
platform	Bloque une chute mais se traverse par le bas
danger	Déclare un contact dangereux pour le jeu
exit	Déclare une sortie ou transition
ladder	Déclare une zone d'échelle

10. Scènes et jeux multi-niveaux

10.1 Une scène par état du jeu

```
c64.game.scene("title", {
  enter() { c64.printCentered(12, "FIRE TO START"); },
  update() {
    c64.control.if(joystick.firePressed(), () => c64.game.go("game"));
  }
});

c64.game.scene("game", {
  enter() { level1.activate({ draw: true }); },
  update() { player.update(); }
});

c64.game.start("title", { hz: 50 });
```

enter() est exécuté à l'entrée de la scène. update() est sa boucle. game.go() programme une transition sûre entre deux frames.

10.2 Stratégie mémoire D64

- Gardez le joueur, les routines communes et l'interface dans le PRG principal.
- Placez les maps et sprites propres à chaque niveau en assets non résidents.

- Réutilisez les mêmes adresses RAM pour les ressources qui ne coexistent jamais.
- Chargez le niveau suivant pendant une transition ou un écran noir.
- Consultez le rapport de build pour vérifier les noms de fichiers disque et les plages mémoire.

Sur une vraie disquette

Les noms C64 sont courts et normalisés. Utilisez les noms produits par le compilateur et ne renommez pas manuellement les modules de données dans l'image D64.

11. Son SID

11.1 Réglages directs

Fonction	Rôle
c64.sid.volume(0..15)	Volume global
c64.sid.voice(1..3)	Sélection d'une voix
voice.frequency(valeur)	Fréquence SID brute
voice.pulseWidth(valeur)	Largeur d'impulsion
voice.waveform(type)	triangle, sawtooth, pulse ou noise
voice.attackDecay(v)	Enveloppe attaque/décroissance
voice.sustainRelease(v)	Enveloppe maintien/relâchement
voice.gate(true/false)	Démarre ou arrête la note

11.2 Musique et bruitages non bloquants

```
c64.sid.volume(12);
c64.sid.playSong({
  tempo: 6,
  voices: [
    ["C4", "E4", "G4", "E4"],
    ["C3", "R", "G2", "R"],
    ["R", "C5", "R", "G4"]
  ]
});

c64.sid.click();
```

Le lecteur musical et les effets sont pilotés par IRQ ou par un runtime partagé. Les effets prêts à l'emploi ne doivent pas arrêter la balle, le joueur ou la boucle de jeu.

- Réservez une voix aux effets dans un jeu avec musique.
- Utilisez les instruments et patterns pour réduire les tables répétées.
- Consultez la section **sid-audio** du rapport de build pour les conflits de voix.

12. IRQ raster et assembleur bas niveau

12.1 IRQ raster

```
c64.irq.raster(100, () => {
  c64.borderColor(c64.COLOR_RED);
});

c64.irq.install();
```

API	Utilité
c64.irq.raster(ligne, fn)	Ajoute une action à une ligne raster
c64.irq.rasterLoop(ligne, fn, options)	Effet raster répété et stabilisé
c64.irq.install()	Installe le dispatcher IRQ
c64.irq.chainToKernal()	Conserve le traitement IRQ du KERNAL
c64.irq.disableKernalTimer()	Évite les IRQ CIA concurrentes dans un moteur raster
c64.irq.ack()	Acquitte manuellement une IRQ dans un usage avancé

12.2 Assembleur 6502 dans le JavaScript

```
c64.asm.label("change_border");
c64.asm.lda(c64.imm(c64.COLOR_BLUE));
c64.asm.sta(c64.abs(c64.VIC_BORDER_COLOR));
c64.asm.rts();
```

Les helpers d'adressage les plus courants sont `imm()`, `abs()`, `absX()`, `zp()` et `rel()`. Réservez cette couche aux effets ou routines absents de l'API haut niveau.

Conseil

Commencez par l'API de jeu. Descendez vers `c64.asm` seulement après avoir isolé une routine courte et mesuré son intérêt.

13. Optimisation et débogage

13.1 Choisir le profil

Profil	Quand l'utiliser
balanced	Choix par défaut : bon compromis taille/vitesse
size	Quand la taille du PRG ou des modules D64 est prioritaire
speed	Quand le temps CPU ou le démarrage compte plus que la taille

```
npm run build -- --platformer-mini.js `
-o .\build\platformer-mini.prg `
--opt balanced --report .\build\platformer-mini-report.json
```

13.2 Lire les sorties de diagnostic

- **ASM** : routines et labels générés.
- **LST** : adresse exacte, octets et instruction de chaque ligne.
- **symbols.json** : adresses des labels pour le moniteur VICE.
- **report.json** : taille, plages RAM, RLE, IRQ, sprites, maps, SID et conflits.

13.3 Erreurs courantes

Message ou symptôme	Vérification
c64js n'est pas reconnu	Utiliser node .\src\cli.js ou npx c64js
Cannot find module src/cli.js	Vérifier Get-Location et revenir dans le bon dossier
Le fichier source est introuvable	Tester Get-ChildItem .\examples\nom.js
Le jeu ne réagit pas	Vérifier le port joystick, game.frame() et player.update()
PC bloqué dans VICE	Ouvrir le LST à l'adresse du PC et inspecter la boucle
Disk error	Reconstruire la D64 et ne pas renommer ses modules
Sprite coupé ou absent	Vérifier Y, multiplexage et limite de 8 sprites par raster
Collision décalée	Vérifier hitbox, offset et vitesse maximale de collision

14. Référence rapide des fonctions

Cette référence regroupe les fonctions les plus utiles. Les options avancées et signatures exhaustives restent documentées dans **MODE_EMPLOI_DEBUTANT.txt** et **README.md**.

14.1 Écran et système

Fonction	Résumé
clearScreen	Effacer l'écran
borderColor	Couleur de bordure
backgroundColor	Couleur de fond
textColor	Couleur texte
print / printAt / printCentered	Afficher du texte
waitKey	Attendre une touche dans un programme simple
writeChar	Écrire un code écran
fillRect / drawFrame / clearLine	Dessiner l'interface texte
screen / colorRam	Changer les adresses vidéo utilisées
sys	Appeler une adresse machine
label / comment	Annoter le code généré

14.2 Données, variables et contrôle

Famille	Fonctions principales
c64.data	byte, word, string, screenString, length
c64.var	byte, word, bool
c64.table	byte et accès indexé runtime
c64.control	if, while, routine, call
c64.game	init, frame, every, scene, go, start, score, lives
c64.random	seed, range
c64.pool	fixed

14.3 Entrées, sprites et collisions

Famille	Fonctions principales
c64.input.joystick	left, right, up, down, fire, firePressed, fireReleased
c64.input.keyboard	touches nommées, down, pressed, released
c64.sprite	create, position, color, enable, disable
sprite logique	setPosition, setVelocity, update, play, collides, respawn
collision map	moveAndCollide, contacts, collisionBehaviors

14.4 Assets, maps et caméra

Fonction	Résumé
assets.loadMap / defineMap	Charger ou définir une map
assets.loadSprite / defineSprite	Charger ou définir un sprite asset
charset.use	Installer un charset RAM avec caractères ROM 0-63 automatiques
map.draw / level.map(x,y)	Dessiner, lire et modifier les tiles
level.activate / isActive	Changer de niveau en inline ou D64
map.spawn	Créer une entité depuis un objet de map
map.scroller	Créer une caméra X/Y avec panneau IRQ
camera.draw / follow / left / right / up / down	Dessiner, suivre et déplacer la caméra

14.5 Son, IRQ et assembleur

Famille	Fonctions principales
c64.sid	volume, filter, voice, note, rest, playSong, stopSong, click, effets
c64.irq	raster, rasterLoop, install, chainToKernal, disableKernalTimer, ack
c64.asm	lda, sta, adc, sbc, and, ora, eor, cmp, branches, jmp, jsr, rts, byte, label
adressage	imm, abs, absX, absY, zp, zpX, ind, indY, rel, immLo, immHi

15. Méthode conseillée pour votre premier jeu

15.1 Progression en sept petites étapes

Étape	Objectif vérifiable
1	Afficher un écran fixe et produire PRG + ASM
2	Lire le joystick dans game.frame()
3	Déplacer un sprite avec limites
4	Ajouter une collision et un score
5	Ajouter une map créée dans le Studio
6	Ajouter scrolling, panneau fixe et son
7	Séparer title, game et gameOver, puis produire une D64 si nécessaire

15.2 Checklist avant de continuer

- Le PRG se lance dans VICE sans blocage.
- Le joystick est lu sur le bon port.
- La boucle reste courte et toutes les boucles runtime sont bornées.
- Les coordonnées, hitboxes et collisions sont testées aux quatre directions.
- Le texte reste lisible avec le charset personnalisé.
- Le rapport ne signale aucun conflit mémoire ni dépassement raster.
- Les sorties ASM et LST sont conservées avec chaque build important.

Le meilleur réflexe

Ajoutez une seule fonction à la fois, compilez, testez dans VICE, puis seulement ensuite passez à l'étape suivante. C'est la manière la plus rapide de comprendre où apparaît un défaut.

Fin du guide condensé. Pour les signatures complètes, les contraintes exactes et les exemples spécialisés, consultez les fichiers **MODE_EMPLOI_DEBUTANT.txt**, **README.md** et le dossier **examples/** du projet.